

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR
Filière : 4^{ème} année — Intelligence Artificielle et Data
Module : Deep Learning

SYNTHÈSE DU PROJET DE FIN DE MODULE

Conception, implémentation, comparaison et analyse critique
de modèles de deep learning

Réalisée par :
Imane Nadif

Encadré par :
Pr. Mosaab Batal

Groupe :
G2 — Moulay Youssef

Année universitaire 2025–2026

Table des matières

1	Vue d'ensemble	3
2	Partie I — MLP et ingénierie PyTorch	3
2.1	Concepts fondamentaux	3
2.1.1	nn.Module, paramètres, gradient, state_dict, device	3
2.2	Dataset : Wine Quality UCI	4
2.3	Implémentation : nn.Sequential vs classe personnalisée	4
2.4	Trois stratégies d'initialisation	4
2.5	Test des 5 architectures et sélection du Best Model	5
2.6	Évaluation du Best Model	6
2.7	Question de synthèse — Partie I	7
3	Partie II — CNN et architectures dérivées	8
3.1	Limites du MLP pour les images	8
3.2	Les 3 idées fondatrices des CNN	8
3.3	Corrélation croisée 2D et calculs manuels	8
3.4	Canaux multiples	8
3.5	Convolution 1×1	9
3.6	Architectures CNN implémentées	9
3.6.1	LeNet	9
3.6.2	AlexNet simplifié	9
3.6.3	VGG simplifié	9
3.7	Visualisation et interprétation des feature maps	9
3.8	Comparaison MLP vs CNN vs Dérivés	10
3.9	Question de synthèse — Partie II	11
4	Partie III — RNN / LSTM / GRU / Transformers	11
4.1	Modèles de langage	11
4.1.1	Objectif probabiliste et règle de chaîne	11
4.1.2	Approches n-grammes	11
4.1.3	Perplexité	11
4.2	Réseaux de neurones récurrents (RNN)	11
4.2.1	Idée centrale	11
4.2.2	BPTT et gradient clipping	12
4.3	LSTM : Long Short-Term Memory	12
4.3.1	Pourquoi les LSTM ?	12
4.3.2	Équations	13
4.3.3	Rôle des portes	13
4.4	GRU : Gated Recurrent Unit	13
4.4.1	Principe	13
4.4.2	Comparaison LSTM / GRU	13
4.5	Variantes architecturales : RNN profonds et bidirectionnels	13
4.5.1	RNN profonds	13
4.5.2	Dropout récurrent	13
4.5.3	RNN bidirectionnels	14
4.6	Préparation des données — Tatoeba fra-eng	14
4.7	Architecture encodeur-décodeur	14

4.7.1	Idée générale	14
4.8	Décodage : glouton et beam search	15
4.9	Transformer — <i>Attention is all you need</i>	15
4.9.1	Mécanisme d'attention	15
4.9.2	Positional encoding	15
4.10	Évaluation : Perplexité et BLEU	15
4.11	Comparaison globale des modèles séquentiels	16
4.12	Question de synthèse — Partie III	17
5	Question transversale finale	17
5.1	Tableau de synthèse général	17
5.2	Conclusion	17

1. Vue d'ensemble

Ce projet couvre trois grandes familles d'architectures de deep learning, chacune adaptée à un type de données spécifique. La progression suit une logique pédagogique claire : données tabulaires (MLP), données structurées spatialement (CNN + dérivés), puis données séquentielles (RNN, LSTM, GRU, Transformers).

Une chaîne de traitement typique est la suivante :

1. Représenter les données sous forme de tenseurs
2. Définir un modèle adapté à la structure des données
3. Entraîner le modèle par optimisation d'une perte
4. Évaluer et interpréter les résultats
5. Comparer les architectures de manière critique

Principe unificateur

Un même principe (descente de gradient, rétropropagation) s'applique à toutes les architectures. Ce qui change est l'**inductive bias** : la façon dont chaque architecture exploite la structure géométrique, locale ou temporelle des données.

Partie	Architecture	Type de données	Dataset
I	MLP	Tabulaires	Wine Quality UCI (1 599 exes)
II	CNN + Dérivés	Images	MNIST (70 000 images 28×28)
III	RNN / LSTM / GRU / Transformer	Séquences	Tatoeba fra-eng (5 000 paires)

2. Partie I — MLP et ingénierie PyTorch

2.1. Concepts fondamentaux

2.1.1. nn.Module, paramètres, gradient, state_dict, device

En PyTorch, tout réseau hérite de `nn.Module`. Un **paramètre** est un tenseur avec `requires_grad=True`. Lors de `loss.backward()`, les gradients sont calculés via la règle de la chaîne :

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial W}, \quad W \leftarrow W - \eta \cdot \frac{\partial \mathcal{L}}{\partial W}$$

Propagation avant :

$$x \xrightarrow{W_1, b_1} z_1 \xrightarrow{\text{ReLU}} a_1 \xrightarrow{W_2, b_2} z_2 \xrightarrow{\text{ReLU}} a_2 \xrightarrow{W_3, b_3} \hat{y}$$

Le `state_dict()` est un dictionnaire ordonné de tous les poids qui permet de sauvegarder et recharger le meilleur modèle. Le modèle **et** les données doivent être sur le même device (CPU ou GPU). Le **Dropout** désactive aléatoirement une fraction p des neurones pendant l'entraînement pour réduire le surapprentissage.

2.2. Dataset : Wine Quality UCI

Le dataset utilisé est le **Wine Quality UCI** (vins rouges), qui contient **1 599 exemples**, **11 features** chimiques (acidité, sucre, alcool, pH, etc.) et **6 classes** correspondant aux notes de qualité de 3 à 8. Ce dataset est plus réaliste et plus challenging que le dataset sklearn original (178 exemples, 3 classes), notamment à cause du **déséquilibre des classes** : les notes extrêmes (3, 4, 8) sont très peu représentées.

Le pipeline de préparation comprend : normalisation **StandardScaler** (ajustée uniquement sur le train), split stratifié train / val / test, et DataLoaders de taille 32.

2.3. Implémentation : nn.Sequential vs classe personnalisée

Deux approches ont été implémentées :

- **nn.Sequential** : approche rapide qui empile des couches linéairement. Adaptée aux architectures simples sans branchements. Le modèle prend 11 features en entrée et produit 6 sorties.
- **Classe personnalisée (MLP)** : hérite de **nn.Module** et offre plus de flexibilité — choix de l'activation (ReLU, Tanh, Sigmoid), taux de Dropout paramétrable, nombre de couches variable.

2.4. Trois stratégies d'initialisation

Stratégie	Formule	Analyse
Gaussienne	$W \sim \mathcal{N}(0, 0.01)$	Convergence lente (poids trop petits)
Constante	$W = 1.0$	Symétrie impossible à briser
Xavier	$W \sim U\left(-\frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}}, +\frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}}\right)$	Stabilise la variance — recommandé

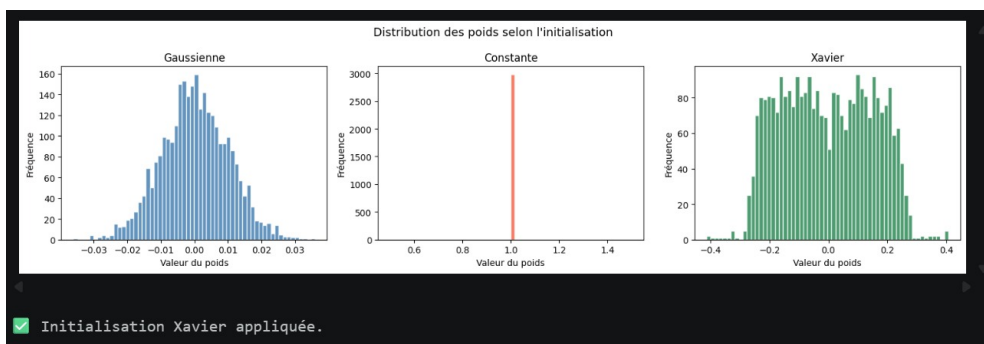


FIGURE 1 – Distribution des poids : Gaussienne vs Constante vs Xavier

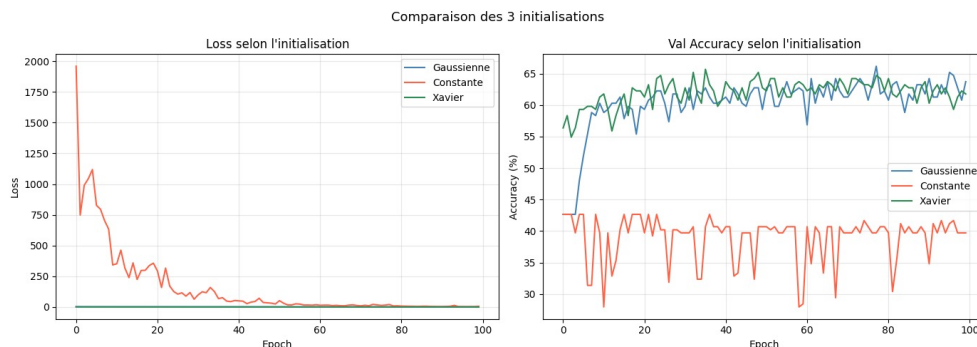


FIGURE 2 – Comparaison des 3 initialisations — Loss et Val Accuracy sur le Best Model

Analyse des résultats : L'initialisation **Xavier** converge plus rapidement et atteint les meilleures performances. La **Gaussienne** converge aussi mais plus lentement car les poids de départ sont trop petits. La **Constante** échoue à cause du problème de symétrie : tous les neurones reçoivent les mêmes gradients et apprennent la même chose, ce qui se traduit par une loss initiale très élevée (~ 2000) et une val accuracy instable et oscillante.

2.5. Test des 5 architectures et sélection du Best Model

Cinq architectures MLP de complexité croissante ont été testées sur le dataset Wine Quality UCI avec l'initialisation Xavier :

Architecture	Couches cachées	Paramètres	Val Acc.	Test Acc.
MLP 1	[32]	582	60.29%	62.50%
MLP 2	[64, 32]	3 046	63.24%	67.50%
MLP 3	[128, 64, 32]	12 070	62.75%	63.33%
MLP 4	[256, 128, 64]	44 614	66.67%	68.75%
MLP 5	[512, 256, 128, 64]	179 014	65.69%	63.33%

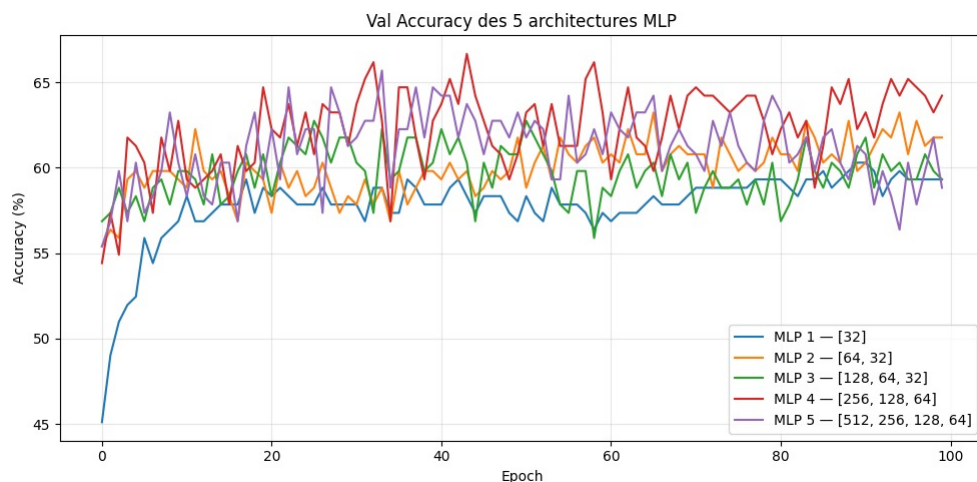


FIGURE 3 – Comparaison Val Accuracy des 5 architectures MLP (Wine Quality UCI)

2.6. Évaluation du Best Model

Métrique	Valeur	Interprétation
Accuracy	68.75%	Proportion de prédictions correctes
Precision	34.22%	Parmi les prédictions positives, combien sont correctes ?
Recall	35.08%	Parmi les vrais positifs, combien sont détectés ?
F1-score	34.64%	Moyenne harmonique precision/recall

Analyse : L'accuracy globale de 68.75% est cohérente avec la difficulté du dataset. Les classes majoritaires (qualité 5, 6, 7) sont bien prédites avec des taux de réussite entre 63% et 79%. En revanche, les classes rares (qualité 3, 4, 8) ne sont jamais prédites correctement à cause du **déséquilibre des classes**. La précision macro et le rappel macro sont faibles (34%) car ces métriques moyennent toutes les classes sans tenir compte du déséquilibre.



FIGURE 4 – Courbes Loss et Val Accuracy du Best Model : MLP 4 — [256, 128, 64]

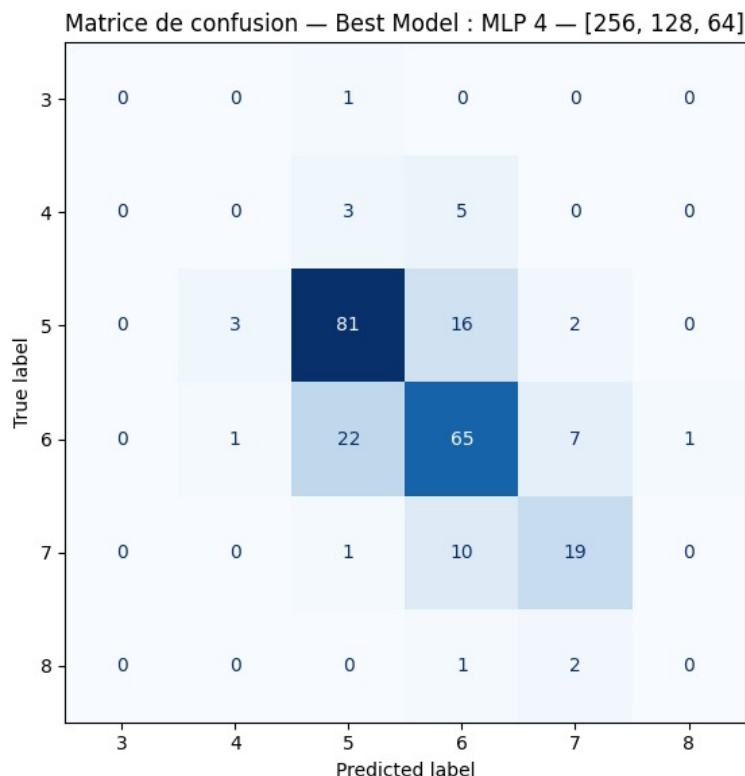


FIGURE 5 – Matrice de confusion du Best Model (MLP 4 — [256, 128, 64]) sur le test set

Lecture de la matrice de confusion : La diagonale représente les bonnes prédictions. On observe que les classes 5 et 6 concentrent la quasi-totalité des prédictions correctes (81/102 et 65/96 respectivement). Les classes 3, 4 et 8 ont une précision nulle : le modèle les classe systématiquement en 5 ou 6, faute d'exemples d'entraînement suffisants.

2.7. Question de synthèse — Partie I

Question

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire, et quelles sont ses principales limites ?

Le MLP est bien adapté aux données tabulaires car les features n'ont pas de structure spatiale ni temporelle. Sur le dataset Wine Quality UCI (1 599 exemples, 11 features, 6 classes), le Best Model est l'architecture MLP 4 — [256, 128, 64] avec une test accuracy de 68.75%. Ce résultat, bien qu'inférieur aux performances obtenues sur le dataset sklearn (3 classes, données plus simples), est représentatif de la réalité : un dataset déséquilibré avec des classes rares est intrinsèquement difficile à classifier.

L'initialisation **Xavier** s'est révélée la plus efficace car elle stabilise la variance des activations à travers les couches. L'initialisation constante échoue à cause du problème de symétrie, et la gaussienne converge plus lentement à cause de poids initiaux trop petits.

Limites :

- Déséquilibre des classes : les classes rares (3, 4, 8) sont ignorées
- Aucun inductive bias structurel : le MLP traite toutes les features de manière indépendante
- Inadapté aux images et aux séquences
- Piste d'amélioration : utiliser une **loss pondérée** ou **SMOTE** pour rééquilibrer les classes

3. Partie II — CNN et architectures dérivées**3.1. Limites du MLP pour les images**

Image MNIST $28 \times 28 = 784$ pixels. Si on aplatit : perte de la structure spatiale, explosion des paramètres ($784 \times 256 = 200\,704$), pas d'invariance aux translations.

3.2. Les 3 idées fondatrices des CNN

1. **Localité** : le filtre ne regarde qu'une petite région
2. **Partage des poids** : le même filtre glisse sur toute l'image
3. **Hiérarchie** : bords $\rightarrow$ formes $\rightarrow$ objets

3.3. Corrélation croisée 2D et calculs manuels

$$S(i, j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} X(i+m, j+n) \cdot K(m, n), \quad H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K + 2P}{S} \right\rfloor + 1$$

Exemple numérique :

$$X = \begin{pmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 2 & 2 \\ 0 & 1 & 3 & 1 \\ 2 & 2 & 1 & 0 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \Rightarrow \text{Sortie} = \begin{pmatrix} 2 & 4 & 2 \\ 4 & 4 & 3 \\ 2 & 2 & 3 \end{pmatrix}$$

Tableau des tailles de sortie (entrée 28×28 , kernel 5×5) :

padding	stride	H_{out}	Calcul
0	1	24	$\lfloor (28 - 5)/1 \rfloor + 1$
2	1	28	$\lfloor (28 - 5 + 4)/1 \rfloor + 1$
0	2	12	$\lfloor (28 - 5)/2 \rfloor + 1$
2	2	14	$\lfloor (28 - 5 + 4)/2 \rfloor + 1$

Taille après max-pooling 2×2 , stride= 2 : $\lfloor (28 - 2)/2 \rfloor + 1 = 14$.

3.4. Canaux multiples

Avec C_{in} canaux et C_{out} filtres : $S(i, j) = \sum_c \sum_m \sum_n X_c(i+m, j+n) \cdot K_c(m, n)$.
Paramètres (sans biais) : $C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w$.

3.5. Convolution 1×1

Opère sur tous les canaux sans toucher la taille spatiale. Réduit le nombre de canaux, ajoute de la non-linéarité. Utilisé dans Inception et ResNet.

3.6. Architectures CNN implémentées

3.6.1. LeNet

Couche	Entrée	Opération	Sortie
Conv1	[B,1,28,28]	Conv2d(1→6, k=5, p=2)+ReLU	[B,6,28,28]
Pool1	[B,6,28,28]	MaxPool2d(2,2)	[B,6,14,14]
Conv2	[B,6,14,14]	Conv2d(6→16, k=5)+ReLU	[B,16,10,10]
Pool2	[B,16,10,10]	MaxPool2d(2,2)	[B,16,5,5]
FC1	[B,400]	Linear(400→120)+ReLU	[B,120]
FC2	[B,120]	Linear(120→84)+ReLU	[B,84]
FC3	[B,84]	Linear(84→10)	[B,10]

3.6.2. AlexNet simplifié

Ajout de **BatchNorm** (stabilise l'entraînement), **Dropout** $p = 0.5$ (réduit le surapprentissage), plus de filtres (32, 64, 128).

3.6.3. VGG simplifié

Blocs de 2 convolutions 3×3 empilées avant chaque pooling. Deux conv 3×3 couvrent le même champ récepteur qu'une conv 5×5 avec moins de paramètres et plus de non-linéarité.

3.7. Visualisation et interprétation des feature maps

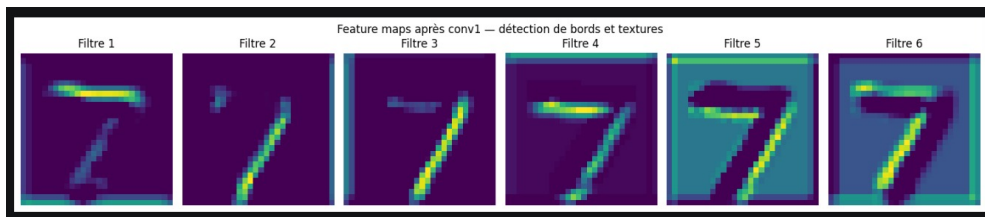


FIGURE 6 – Feature maps après la première couche de convolution (6 filtres, 28×28)

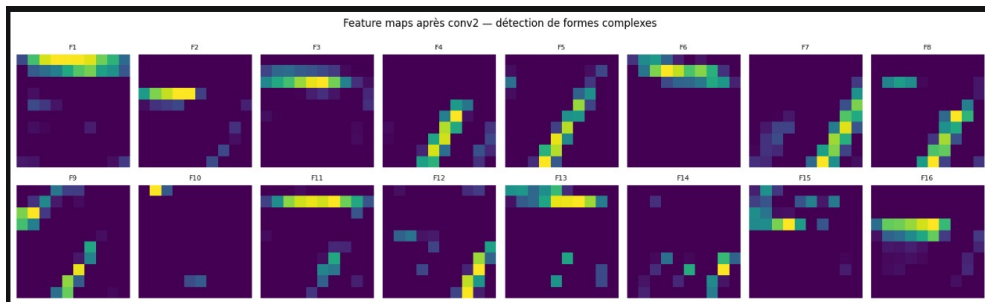


FIGURE 7 – Feature maps après la deuxième couche de convolution (16 filtres, 10×10)

Hiérarchie des représentations

conv1 = bas niveau (bords et contours) → conv2 = niveau intermédiaire (coins, courbes) → couches FC = niveau sémantique (classe).

3.8. Comparaison MLP vs CNN vs Dérivés

Modèle	Paramètres	Test Accuracy
MLP	235 146	97.45%
LeNet	61 706	98.89%
AlexNet simplifié	1 701 322	99.30%
VGG simplifié	870 634	99.28%

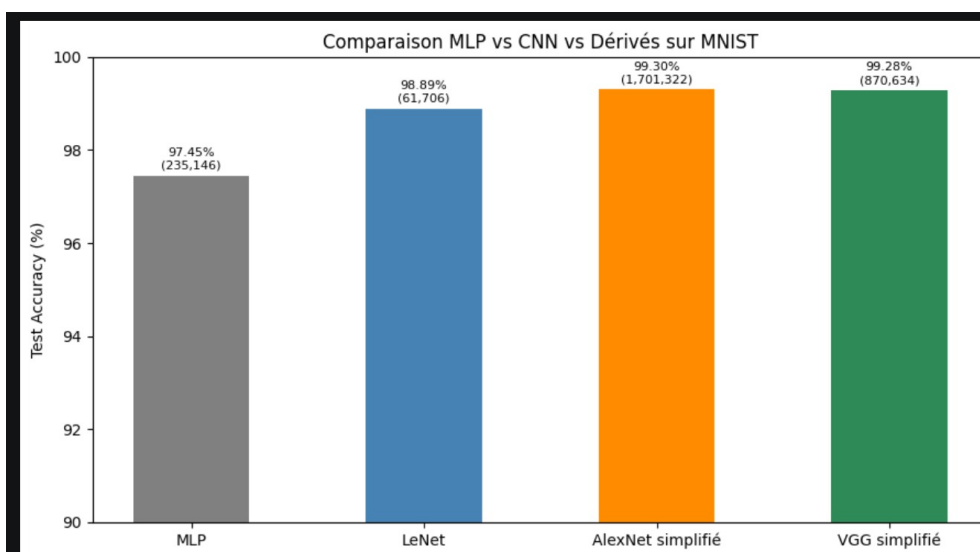


FIGURE 8 – Comparaison Test Accuracy : MLP vs CNN vs Dérivés (MNIST)

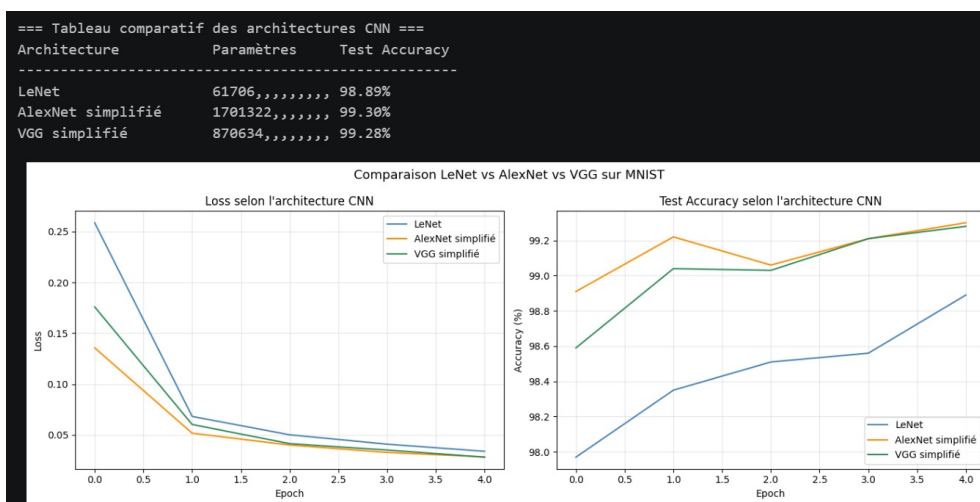


FIGURE 9 – Courbes d'apprentissage : LeNet vs AlexNet vs VGG

3.9. Question de synthèse — Partie II

Question

Pourquoi un CNN est-il plus pertinent qu'un MLP pour les images, et comment les architectures dérivées améliorent-elles les performances ?

Le CNN exploite la structure spatiale. Sur MNIST il obtient 98.89% avec 61 706 paramètres contre 97.45% pour le MLP. AlexNet ajoute BatchNorm et Dropout : 99.30%. VGG empile des conv 3×3 : 99.28%. Le max-pooling conserve les activations dominantes. La conv 1×1 réduit les canaux sans coût spatial.

4. Partie III — RNN / LSTM / GRU / Transformers

4.1. Modèles de langage

4.1.1. Objectif probabiliste et règle de chaîne

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1})$$

Pour modéliser une séquence entière, il suffit de prédire le prochain token à partir du passé.

4.1.2. Approches n-grammes

Dans les modèles de Markov d'ordre $n-1$: $P(x_t \mid x_1, \dots, x_{t-1}) \approx P(x_t \mid x_{t-n+1}, \dots, x_{t-1})$.

Bigramme : $P(x_1, \dots, x_T) \approx P(x_1) \prod_{t=2}^T P(x_t \mid x_{t-1})$.

Limites : explosion combinatoire, sparsité, incapacité à capturer de longues dépendances $\rightarrow$ d'où l'intérêt des RNN.

4.1.3. Perplexité

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log P(x_t \mid x_{<t})\right)$$

Plus la perplexité est faible, meilleur est le modèle.

4.2. Réseaux de neurones récurrents (RNN)

4.2.1. Idée centrale

Un RNN possède un état caché qui se transmet de pas en pas :

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h), \quad \hat{y}_t = \text{softmax}(W_{hq} h_t + b_q)$$

4.2.2. BPTT et gradient clipping

La BPTT déroule le RNN sur T pas. Le gradient d'une perte finale par rapport à un état antérieur contient une chaîne de jacobiens :

$$\frac{\partial L}{\partial h_k} = \sum_{t \geq k} \frac{\partial L}{\partial h_t} \prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}}$$

Gradient évanescent : le produit tend vers zéro. **Gradient explosif** : le produit devient très grand.

Gradient clipping : $g \leftarrow \min\left(1, \frac{\theta}{\|g\|}\right) g$

BPTT tronquée : au lieu de rétropropager sur toute la séquence, on coupe l'horizon temporel après τ pas. Réduit le coût mémoire au prix d'une vision plus courte du passé.

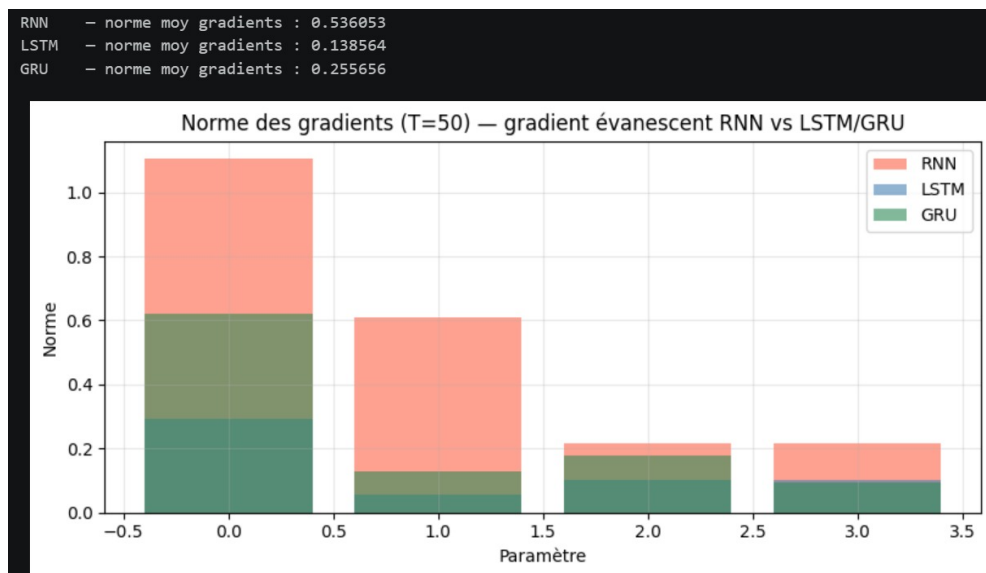


FIGURE 10 – Norme des gradients (T=50) : gradient évanescent RNN vs LSTM/GRU

4.3. LSTM : Long Short-Term Memory

4.3.1. Pourquoi les LSTM ?

Les LSTM ont été conçus pour conserver l'information utile sur de longues distances. Ils introduisent un état de cellule c_t et des portes contrôlant l'écriture, l'oubli et la lecture.

4.3.2. Équations

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{porte d'oubli}) \quad (1)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{porte d'entrée}) \quad (2)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (\text{candidat}) \quad (3)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{cellule mémoire}) \quad (4)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{porte de sortie}) \quad (5)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{état caché}) \quad (6)$$

4.3.3. Rôle des portes

f_t : quelle mémoire garder. i_t : quoi écrire. o_t : quoi exposer. La structure réduit l'évanouissement via c_t .

4.4. GRU : Gated Recurrent Unit

4.4.1. Principe

Le GRU simplifie le LSTM. Pas d'état de cellule séparé. Deux portes :

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad (\text{mise à jour}) \quad (7)$$

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad (\text{réinitialisation}) \quad (8)$$

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h) \quad (9)$$

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t \quad (10)$$

4.4.2. Comparaison LSTM / GRU

Critère	LSTM	GRU
Structure	3 portes + cellule c_t	2 portes
Mémoire	Explicite via c_t	Intégrée à h_t
Coût	Plus élevé	Plus léger
Usage	Longues dépendances	Bon compromis

4.5. Variantes architecturales : RNN profonds et bidirectionnels

4.5.1. RNN profonds

Plusieurs couches récurrentes empilées. À chaque instant t :

$$h_t^{(1)} = f^{(1)}(x_t, h_{t-1}^{(1)}), \quad h_t^{(\ell)} = f^{(\ell)}(h_t^{(\ell-1)}, h_{t-1}^{(\ell)}), \quad \ell \geq 2$$

4.5.2. Dropout récurrent

Dropout entre couches pour limiter le surapprentissage.

4.5.3. RNN bidirectionnels

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

Utile pour l'étiquetage. **Inadapté** à la génération auto-régressive.

4.6. Préparation des données — Tatoeba fra-eng

Dataset réel

Tatoeba fra-eng : corpus parallèle anglais–français. 5 000 paires chargées (longueur ≤ 10 mots), nettoyées, découpées en train / val / test.

Pipeline général :

1. Nettoyage et normalisation des phrases
2. Tokenisation au niveau mot
3. Construction des vocabulaires source et cible
4. Ajout des tokens spéciaux : `<pad>` (0), `<bos>` (1), `<eos>` (2), `<unk>` (3)
5. Padding et masquage : `CrossEntropyLoss(ignore_index=0)`
6. Création des mini-lots

4.7. Architecture encodeur–décodeur

4.7.1. Idée générale

$$P(y_1, \dots, y_{T_y} \mid x_1, \dots, x_{T_x}) = \prod_{t=1}^{T_y} P(y_t \mid y_{<t}, x_{1:T_x})$$

Encodeur : transforme la séquence source en représentation interne. Décodeur : génère la cible à partir de cette représentation.

Teacher forcing : on fournit le vrai token précédent au décodeur pendant l'entraînement. Stabilise l'apprentissage mais crée un écart avec l'inférence (*exposure bias*).

Fonction de perte masquée : $\mathcal{L} = -\sum_t m_t \log P(y_t^* \mid y_{<t}, x)$, avec $m_t = 0$ sur les positions de padding.

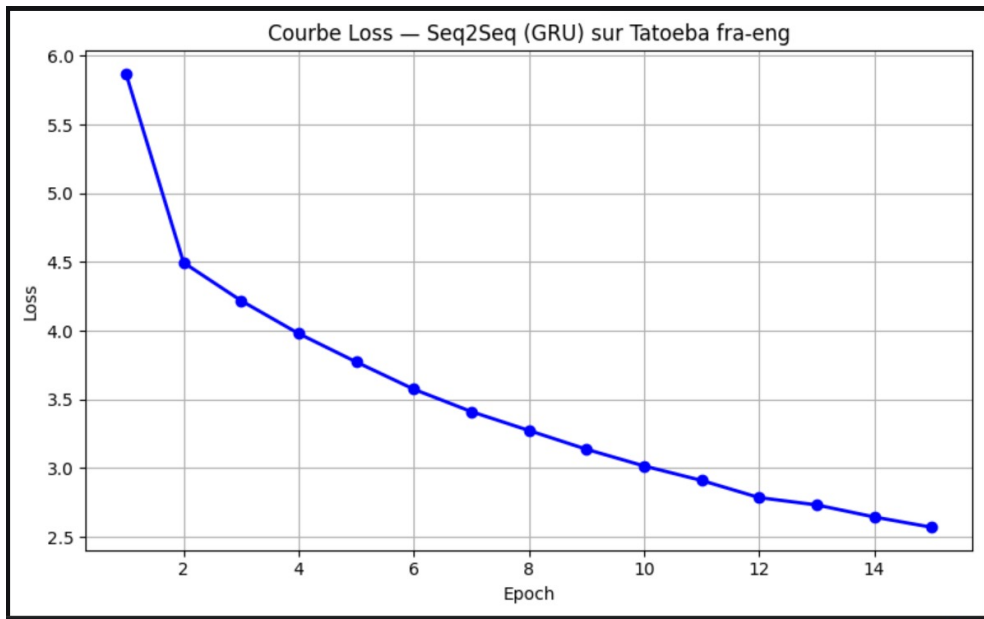


FIGURE 11 – Évolution de la perte — Seq2Seq (encodeur GRU + décodeur GRU)

4.8. Décodage : glouton et beam search

Décodage glouton : $y_t = \arg \max_y P(y \mid y_{<t}, x)$ — Rapide mais sous-optimal.

Beam search : conserve les k meilleures hypothèses partielles. Score normalisé :

$$\text{score}(y) = \frac{1}{T^\alpha} \sum_{t=1}^T \log P(y_t \mid y_{<t}, x), \quad \alpha \in [0, 1]$$

4.9. Transformer — *Attention is all you need*

4.9.1. Mécanisme d'attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

4.9.2. Positional encoding

$$\text{PE}_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

Avantage : capture les dépendances longues sans gradient évanescent. Parallélisable contrairement aux RNN.

4.10. Évaluation : Perplexité et BLEU

Score BLEU :

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right)$$

Modèle	PPL	Temps (s)	Paramètres	Stabilité
RNN	8.31	54.03	531 071	Faible
LSTM	10.20	37.13	605 567	Élevée
GRU	7.71	61.20	580 735	Élevée
Transformer	6.20	n/d	405 311	Très élevée
BLEU — Glouton		0.0256		
BLEU — Beam ($k = 3$)		0.0131		

n/d : temps non mesuré dans la même boucle de chronométrage que les autres modèles.

Remarque

La perplexité LSTM (10.20) est supérieure à celle du RNN (8.31), ce qui est contre-intuitif puisque le LSTM est théoriquement plus performant pour capturer les dépendances longues. Ce résultat s'explique par la taille limitée du dataset (5 000 paires) et la longueur réduite des séquences (≤ 14 tokens) : le RNN simple ne souffre pas significativement du gradient évanescent sur des séquences aussi courtes, alors que le LSTM, plus lourd (605 567 paramètres contre 531 071 pour le RNN), est sur-paramétré par rapport à la difficulté réelle de la tâche et converge moins efficacement en seulement 10 epochs. Le GRU (7.71), avec un nombre de portes intermédiaire, offre le meilleur compromis sur ce dataset limité.

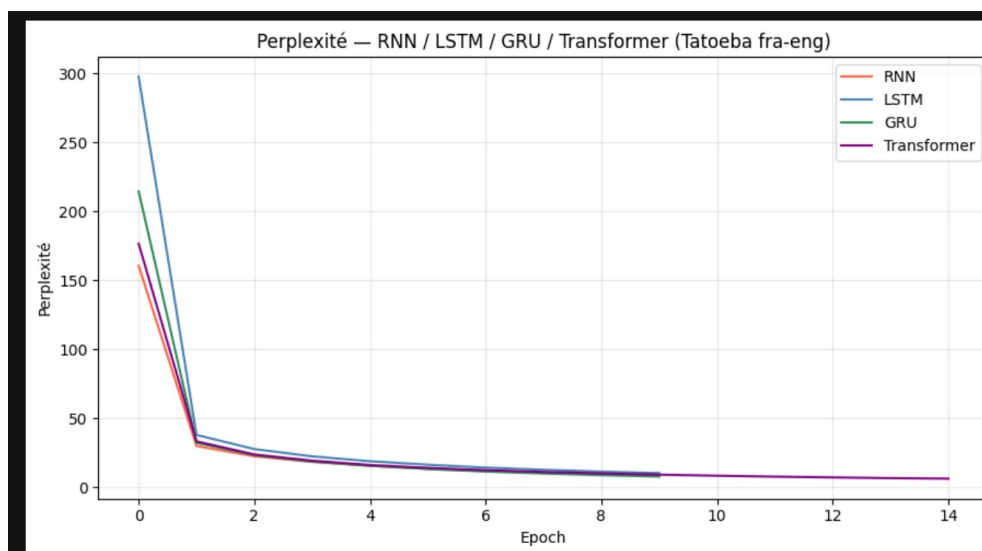


FIGURE 12 – Courbes de perplexité : RNN vs LSTM vs GRU vs Transformer

4.11. Comparaison globale des modèles séquentiels

Modèle	Atouts	Limites
RNN simple	Léger, simple	Gradient évanescent
LSTM	Bon contrôle mémoire	Plus coûteux
GRU	Simple, performant	Moins expressif
Bi-RNN	Utilise passé et futur	Inadapté au décodage causal
Seq2Seq	Modélise source et cible	Goulot sans attention
Beam search	Meilleure exploration	Plus lent
Transformer	Parallélisable, sans évanouissement	Nécessite plus de données

4.12. Question de synthèse — Partie III

Question

Dans quelle mesure les architectures récurrentes permettent-elles de modéliser efficacement une séquence réelle, et comment justifier le passage d'un RNN simple vers un LSTM/GRU puis vers un schéma encodeur-décodeur pour une tâche de traduction ?

Le RNN, sur ces séquences courtes, ne souffre pas fortement du gradient évanescent (PPL = 8.31). Le LSTM, plus complexe et sur-paramétré pour cette tâche, obtient une PPL légèrement supérieure (PPL = 10.20). Le GRU simplifié avec 2 portes (PPL = 7.71). Le Transformer capture les dépendances longues via l'attention directe, sans récurrence (PPL = 6.20). Le beam search améliore légèrement le BLEU : glouton = 0.0256 vs beam = 0.0131. Ce résultat s'explique par la taille limitée du dataset (5000 paires) et le faible nombre d'époques (15).

Limites : le vecteur de contexte fixe du Seq2Seq est un goulot d'étranglement pour les longues phrases — résolu par l'attention.

5. Question transversale finale

Problématique

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle ?

5.1. Tableau de synthèse général

Architecture	Dataset	Inductive bias	Avantage clé
MLP	Wine Quality UCI	Aucun	Flexible
CNN (LeNet)	MNIST	Localité + partage	Invariance spatiale
AlexNet simplifié	MNIST	+ BatchNorm + Dropout	Moins de surapprentissage
VGG simplifié	MNIST	Blocs conv 3×3	Profondeur économique
RNN	Tatoeba	Temporel	Simple, rapide
LSTM / GRU	Tatoeba	Mémoire sélective	Longues dépendances
Transformer	Tatoeba	Attention globale	Parallélisable

5.2. Conclusion

Un même principe (descente de gradient, rétropropagation) s'applique à toutes les architectures. Ce qui change est l'**inductive bias** :

- **MLP** : aucune hypothèse → données tabulaires
- **CNN** : localité spatiale + partage → images
- **RNN / LSTM / GRU** : dépendance temporelle → séquences
- **Transformer** : attention globale → dépendances longues

Cette progression constitue la base des architectures modernes : du MLP classique aux Transformers, chaque famille résout un problème structurel spécifique que la précédente ne pouvait pas traiter efficacement.